

Compiling Agent State

Declarative DAGs for Reliable Multi-Agent Execution

Griffin Downs · Ocean Synaptics AS, General Oceans Group, Oslo, Norway

ABSTRACT Large-language-model agents treat the context window as memory. We externalize agent state into a declarative directed acyclic graph and treat it as a compiled artifact with statically verifiable execution properties. Static analysis provides guarantees before execution—acyclicity, conflict freedom, dependency satisfaction—making parallel dispatch safe by construction. Execution state persists independently of any agent, enabling constant-time recovery and structural convergence across DAG chains. Across 12 active projects over 16 days, we execute 198 DAG iterations comprising 1,794 node completions and 3,425 trail events.

INTRODUCTION

Large-language-model agents typically operate as interpreters: plans, state, and coordination live inside a context window and are reconstructed at runtime. We externalize that state into a declarative directed acyclic graph (DAG)—similar in structure to a Git commit graph, but encoding tasks with dependencies and no cycles. We treat this graph as a compiled artifact with statically verifiable execution properties.

Current systems fail in predictable ways: progress is tied to fragile context windows, parallel execution introduces race conditions, correctness is evaluated after execution rather than before, and iterative runs lack a structural notion of convergence. Compiling agent state into a statically analyzable graph addresses these directly: state persists independently of any agent, parallelism is safe by construction, execution is gated by preconditions, and progress is measured across graph iterations rather than inferred from retries.

Git externalized source code into a persistent DAG, making editors disposable. We apply the same move to agent execution. Coordination shifts from runtime to compile-time, and agents become disposable processes over durable structure.

GRAPH AS EXECUTION STATE

The graph is not only a plan—it is execution state. Nodes encode dependencies and file-level read/write contracts over bounded artifacts. Static analysis establishes invariants before execution: acyclicity, dependency satisfaction, reachability, and conflict freedom. Parallel batches are safe by construction, derived from graph depth. Isolation is enforced through contracts, not negotiated at runtime.

We treat the DAG as a compiled intermediate representation. Task specifications are parsed into a graph IR and subjected to static analysis passes. Execution reduces to a pure function over the graph and a completion predicate, enabling state to be reconstructed and resumed without reliance on prior interaction.

Execution state is persisted via append-only traces, completion receipts, and structured handoffs. The context window is not memory; it is a working cache over durable state.

RECOVERY

When an agent fails, no conversational replay is required. The successor calls `orient()`, receives the graph position, and continues. Agent replacement is a normal runtime event.

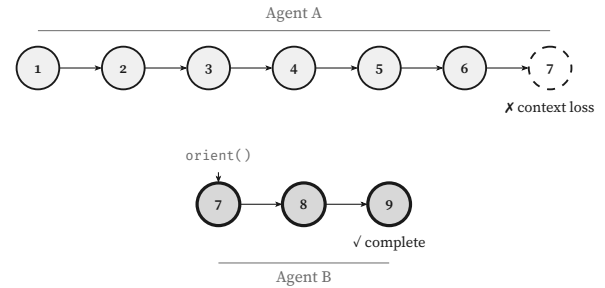


Fig. 1. Agent A fails at node 7. Agent B calls `orient()`, re-executes 7, advances through 9.

CONVERGENCE

DAGs form chains. Each completed graph is mined for residual findings—unaddressed artifacts, weak evidence, or scope drift—and successor graphs are proposed. Trajectory is assessed across iterations (converging, stable, orbiting, diverging), providing a structural notion of progress rather than heuristic retry.

Completion is not declared—it is proven. Nodes advance only when validator evidence passes and receipts are recorded. The analogy to compiler theory is direct: type-checking before execution, not assertion-checking after.

EVALUATION

Across 12 active projects over 16 days:

198 DAG iterations
1,794 node completions
3,425 trail events

The advance rate reflects enforced correction—the system rejects incomplete work until contracts are satisfied. Projects with fewer than 10 advances or 2 DAGs were excluded.

This approach shifts responsibility from runtime coordination to compile-time structure. It does not eliminate runtime isolation in all environments; it reduces reliance on it by enforcing logical isolation statically.

CONCLUSION

The process is ephemeral. The graph is permanent.

The result is a system in which planning, coordination, memory, validation, and convergence collapse into a single substrate. Operational knowledge is carried by the graph, not by the agent.